Security Documentation

ABU-SSO: Unified Single Sign-On & Digital Campus Wallet

Course: COEN838 — Advanced Software Engineering
Student: Idris Muhammad Abubakar
Matric No: P25EGCP9003

---

1. Security Objectives

ABU-SSO protects four classes of assets:

Asset	Confidentiality	Integrity	Availability
User credentials (passwords)	✅	✅	✅
JWT access tokens	✅	✅	✅
Wallet balances & transactions	✅	✅	✅
Digital ID card QR payloads	✅	✅	✅

The system is built on the **CIA Triad** (Confidentiality, Integrity, Availability) plus **non-repudiation** of financial transactions and **least privilege** on account creation.

---

2. STRIDE Threat Model

STRIDE is applied to each major component.

2.1 Authentication Component (`/auth/*`)

Threat	Attack Scenario	Mitigation
Spoofing	Attacker submits stolen credentials	bcrypt password verification; generic error message ("Invalid matric/staff number or password") prevents user enumeration
Spoofing	Attacker replays an old JWT	JWT includes `exp` (15-min expiry) — expired tokens rejected
Tampering	Attacker modifies JWT payload (e.g. change `sub`)	HS256 signature; any tampering invalidates signature → 401
Repudiation	User denies having logged in	Login flow is deterministic; an audit log is listed as future work
Information Disclosure	Passwords leaked via API	Passwords never returned; `password_hash` never serialised

Threat	Attack Scenario	Mitigation
Information Disclosure	Error messages reveal user existence	Login returns one generic message regardless of whether the matric exists
Denial of Service	Attacker floods `/auth/login`	Rate limiting **not yet implemented** (residual risk — see §5)
Elevation of Privilege	Attacker self-registers an admin account	**Bootstrap-only registration**: the very first user on a fresh DB becomes admin; every subsequent call to `/auth/register` returns HTTP 403. New accounts are created by an admin via `POST /admin/users`
Elevation of Privilege	Attacker changes their own role via API	No endpoint allows a normal user to change any user's role; only `PATCH /admin/users/{id}/role` (admin-only) can do so

2.2 Wallet Component (`/wallet/*`)

Threat	Attack Scenario	Mitigation
Spoofing	Attacker calls `/wallet/topup` with another user's token	Token binds to `sub` (user id); every operation scoped to that user
Tampering	Attacker sends negative amount to gain balance	Pydantic `Field(gt=0)` rejects negative amounts with HTTP 422
Tampering	Attacker tries to edit transaction history	`Transaction` records are append-only; no update/delete endpoint exists
Repudiation	User denies a payment	Every transaction stores `amount`, `type`, `description`, `created_at`
Information Disclosure	Attacker enumerates wallet IDs	Wallet is fetched via `user_id` from token — no ID enumeration possible
Denial of Service	Attacker spams top-up endpoint	Not rate-limited (residual risk)
Elevation of Privilege	Attacker tries `pay` on another wallet	Not possible — wallet is always derived from `current_user`

2.3 ID Card Component (`/id-card`)

Threat	Attack Scenario	Mitigation
Spoofing	Attacker forges a QR code	QR encodes user id and matric; verifier must validate against the IdP
Information Disclosure	QR photographed and decoded	QR contains only `{id, matric_no, role}` — no email, password, or wallet info
Tampering	Attacker edits QR payload	Regenerated QR must match the IdP's record

2.4 Admin Component (`/admin/*`)

Threat	Attack Scenario	Mitigation
Elevation of Privilege	Student calls `/admin/users`	Entire router declares `dependencies=[Depends(require_role("admin"))]` — non-admins get HTTP 403
Elevation of Privilege	Attacker creates an admin account via `/admin/users` as a non-admin	Not possible — 403 before the endpoint body runs
Information Disclosure	Non-admin reads user list	403
Tampering	Attacker changes another user's role	Only admins can call this endpoint

2.5 Distributed SSO Services

Threat	Attack Scenario	Mitigation
Spoofing	Attacker crafts a fake token for a service	Services do not decode tokens locally — they call the IdP's `/auth/me` and only trust the IdP's response
Tampering	Attacker edits the token in the URL	Same — signature verification happens at the IdP
Information Disclosure	Token visible in URL (query param)	Prototype limitation; production would use HttpOnly cookies or a BFF pattern
Denial of Service	IdP down	Services return HTTP 503 and serve no data (fail-secure)

2.6 Database Layer

Threat	Attack Scenario	Mitigation
Tampering	SQL Injection via form fields	SQLAlchemy parameterises every query; no string concatenation
Information Disclosure	Attacker reads the DB file	In production, DB is Neon PostgreSQL behind TLS; SQLite file is local-only

3. Secure Design Principles — Applied

Principle	Where Applied
Least Privilege	No user can access another user's wallet/ID card. Only `admin` can create users, change roles, or top up wallets.
Defence in Depth	bcrypt (passwords) + JWT (sessions) + Pydantic (input validation) + SQLAlchemy (parameterised queries) + RBAC (authorization) + Bootstrap-only registration

Principle	Where Applied
	(account provisioning policy) + HTTPS (transport)
Fail Securely	Auth failures return HTTP 401; unauthorized roles return 403; wallet under-funding raises `ValueError` *before* the DB is touched; services return 503 if the IdP is unreachable
Never Trust Input	Every request body is validated by Pydantic schemas *before* it reaches business logic.
Separation of Secrets	`JWT_SECRET` is loaded from `.env` (gitignored). The `.env.example` file contains a placeholder only. In production, Render generates it randomly.
Complete Mediation	Every protected route uses `Depends(get_current_user)` — there is no "back door".
Economy of Mechanism	Authentication logic lives in exactly one place (`app/utils/jwt.py`), reducing the attack surface.
Cryptographic Hygiene	bcrypt cost ≥ 12 by default; JWT secret ≥ 32 bytes (RFC 7518 §3.2).
Least Common Mechanism	No shared state between users; every DB read is filtered by `user_id` from the token.
Bootstrap Pattern	First user on a fresh DB → admin; all subsequent accounts must come from an admin. Prevents self-assigned privilege escalation.

---

4. Authentication & Authorization Scheme

4.1 Authentication — Matric + Password + JWT

- **Identifier:** matric number (students) or staff number (staff). Email is **not** used for authentication.
- **Password storage:** bcrypt with auto-generated salt (default work factor = 12).
- **Token type:** JSON Web Token signed with HMAC-SHA256.
- **Payload claims:**
  - `sub` — user id (UUID)
  - `iat` — issued at (Unix epoch)
  - `exp` — expiry (iat + 900 seconds = 15 minutes)
- **Transport:** `Authorization: Bearer <token>` header (for API), `?token=` query param (for mock services in the demo).
- **Storage on client:** `localStorage["abu_token"]`.

Why JWT and not server-side sessions?

1. **Stateless** — API can scale horizontally.
2. **Cross-service** — six services verify the same token without a shared session store.
3. **Standard** — JWT is the industry norm for SSO.

Why matric instead of email?

- Matches real ABU practice: students memorise their matric number, not which email they used.

- Eliminates a class of user-error login failures.
- Emails are still stored for records (not for auth).

4.2 Password Storage — bcrypt

- **Algorithm:** bcrypt with auto-generated salt (cost = 12).
- **Why bcrypt?** Deliberately slow (~100 ms per hash), resisting brute-force attacks.
- **Verification:** `bcrypt.checkpw()` uses constant-time comparison to mitigate timing attacks.

4.3 Authorization — Role-Based Access Control (RBAC)

Three roles: `student`, `staff`, `admin`. Enforced by a reusable FastAPI dependency factory:

```
def require_role(*allowed_roles: str):
    def _checker(current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in allowed_roles:
            raise HTTPException(403, f"Requires role: {' or
'.join(allowed_roles)}")
        return current_user
    return _checker

router = APIRouter(
    prefix="/admin",
    dependencies=[Depends(require_role("admin"))],
)
Every endpoint in the router is automatically protected. Non-admin callers receive
HTTP 403 Forbidden.

4.4 Account Provisioning — Bootstrap-Only Registration
Situation   Behavior
Database has 0 users    POST /auth/register creates the first user with
role="admin" (bootstrap)
Database has ≥ 1 user   POST /auth/register returns HTTP 403 — public registration
is disabled
Test environment (flag set) POST /auth/register allowed, to let the test suite
create users freely
Why this matters: Without the bootstrap rule, any internet user could call
/auth/register and set role="admin" in the JSON body — a critical privilege-
escalation vulnerability. The bootstrap pattern is the standard fix: allow one-
time provisioning, then lock the door.

Production workflow: students and staff receive their accounts from ABU ICT after
admission or employment, matching the real university process.

5. Security Testing Evidence
5.1 Static Application Security Testing (SAST) — Bandit
Command: bandit -r app/

Result:
Run started: 2026-10

Test results:
```

```
        No issues identified.

Code scanned:
        Total lines of code: 525
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                High:   0
                Medium: 0
                Low:    0

Interpretation: no common Python vulnerabilities (hardcoded passwords, eval, weak
crypto, shell injection) were detected.

5.2 Automated Security Tests (pytest)
The following tests directly verify security properties:
Test    Property Verified
test_hash_password_is_not_plaintext Passwords never stored plaintext
test_hash_password_same_input_different_output  Salting works
test_verify_password_wrong  Wrong passwords rejected
test_login_wrong_password_returns_401   API returns 401, not 200
test_login_unknown_user_returns_401 Unknown user → 401 (no enumeration)
test_me_without_token_returns_401   Missing auth rejected
test_me_with_bad_token_returns_401  Tampered tokens rejected
test_id_card_requires_auth  Protected endpoint requires auth
test_wallet_requires_auth   Protected endpoint requires auth
test_wallet_pay_insufficient_funds_returns_400  Fail-secure on insufficient
balance
test_wallet_pay_insufficient_funds_does_not_change_balance  No partial state
change
test_wallet_negative_topup_returns_422  Input validation rejects negatives
test_admin_endpoints_require_auth   Admin endpoints reject missing tokens
test_admin_endpoints_forbid_students    Students get 403 on admin endpoints
test_student_cannot_create_users    RBAC blocks privilege escalation
test_deactivated_user_cannot_log_in Deactivated accounts rejected
test_student_cannot_access_admin_page (E2E) RBAC enforced at UI level
Total security-relevant tests: 17 — all passing.

5.3 Dependency Audit
All dependencies pinned in requirements.txt. A future release will add pip-audit
to CI:
pip-audit -r requirements.txt

6. Residual Risks & Roadmap
Residual Risk   Mitigation Planned
No rate limiting on /auth/login Add slowapi with 5 attempts/min/IP
No account lockout after N failures Add exponential backoff
No refresh tokens   Add /auth/refresh endpoint
No audit log table  Add audit_logs table (login, wallet ops, admin actions)
Tokens in URL (mock services)   Move to HttpOnly cookies or BFF pattern in
production
No CSRF protection  JWT in Authorization header, not cookies — no CSRF vector
today; revisit if we add cookie-based sessions
```

```
No email verification  Moot — email not used for authentication
No 2FA  Add TOTP (Google Authenticator) as optional 2nd factor
All residual risks are documented in the risk register
(docs/RISK_AND_MAINTENANCE.md) with mitigations and owners.

7. Compliance Mapping
Requirement Source  ABU-SSO Alignment
NDPR (Nigeria Data Protection Regulation)  Data minimisation: only matric, email,
name, role collected. Passwords hashed. QR contains no PII.
OWASP Top 10 (2021) A01 Broken Access Control — mitigated by get_current_user +
require_role; A02 Cryptographic Failures — bcrypt + 32-byte secret; A03 Injection
— parameterised queries; A04 Insecure Design — bootstrap-only registration; A07
Auth Failures — JWT + bcrypt
RFC 7519 (JWT)  Standard claims (sub, iat, exp)
RFC 7518 §3.2 (JWA) HMAC key ≥ 32 bytes
8. Summary of Security Controls
Control Implementation
Password hashing  bcrypt (cost ≥ 12)
Session management  JWT (HS256), 15-min expiry
Login identifier  Matric / staff number (not email)
Account provisioning  Bootstrap-only, then admin-only
Role-based authorization  require_role("admin") middleware
Input validation  Pydantic schemas
SQL injection prevention  SQLAlchemy parameterisation
Secret management  .env gitignored; Render auto-generates
Transport security  HTTPS (Render TLS)
SAST  Bandit (0 findings)
Test coverage of security  17 security-specific tests
```